

Untangling Scattered Business Logic

An Architecture for a Single Source of Truth Across Code, Rules Engines, and Ontologies

Architecture Proposal — Domain-Centric Metadata & Execution-Translation Pipeline

Prepared for internal architecture review

Contents

Executive Summary

Business logic in most enterprises is scattered across legacy code, database triggers, rules engine UIs, and informal ontologies — with no single place that tells you what the system actually does, or why. The result is **logic drift**: the same rule re-implemented three different ways, each drifting further from the others with every change.

This proposal lays out an architecture that fixes the root cause rather than the symptom. Two ideas do the work:

- **A domain-centric Single Source of Truth (SSOT).** Every constraint, rule, and computation is defined once, in a declarative spec, under version control.
- **An explicit execution-translation pipeline.** A CI/CD build step compiles that one spec into whatever each runtime needs — typed code, semantic graph artifacts, or rules-engine XML — instead of forcing every runtime to speak the same language.

The architecture is deliberately **not** a call to replace legacy engines with one universal tool. It treats business logic as a governed, versioned data artifact, and lets code, rules engines, and ontologies keep doing what each does best.

The Problem: Logic Has No Fixed Address

Legacy estates accumulate three failure modes simultaneously, and they compound each other:

- **Duplication.** The same eligibility rule lives in a stored procedure, a Drools ruleset, and a service-layer if-statement — each maintained independently.
- **Silent drift.** A rule changes in one place during an incident fix and is never propagated to the other two.
- **No audit trail.** Rules engine UIs and database triggers are edited directly in production, with no PR, no review, and no version history.

The instinctive fix — pick one tool and force everything into it — reliably fails. Procedural algorithms don't belong in a rules engine, and stateful workflows don't belong in an ontology. Section 1 gives each kind of logic its correct home before anything is centralized.

1. Classify Logic by Layer — the “Right Home” Principle

Before centralizing anything, classify what kind of logic you're looking at. Mixing these categories — running procedural code inside an ontology, or building stateful algorithms inside a rules engine — is the single largest driver of legacy bloat.

Classify Logic by Layer — The "Right Home" Principle



Figure 1. Three logic categories, each routed to the tool built for its paradigm.

Constraints (structural / semantic)	What must always be true about the domain data?	Ontologies (OWL/RDF); schema definitions (JSON Schema, OpenAPI, Pydantic, Protobuf)	An Order must have at least one Line Item, and the Customer must exist.
Business Rules (policy / decision)	What actions or decisions occur based on state?	Rules engines (Drools, Camunda, DMN) or a DSL	If customer tier is Gold and order total > \$100, apply a 15% discount.
Computational Logic (procedural / algorithmic)	How is state computed or processed sequentially?	Code — microservices, DDD domain services	Route optimization, or a multi-step ML feature pipeline.

2. Treat Logic as Code — the Governance Pipeline

Once logic is classified, it needs a **Single Source of Truth**: a central repository holding logic specs in human-readable form (YAML, DMN, Turtle/OWL). The hard rule that makes this work: **nobody edits a rule directly in a Rules Engine UI or a production database**. Everything is checked into source control first, and reaches production only by passing through CI.

Continuous Integration for Logic

Rule and schema changes go through the same discipline as application code:

- **Linting & validation.** Semantic reasoners (e.g., HermiT) catch contradictory ontology axioms; static analysis catches malformed or conflicting DMN tables before merge.

- **Automated unit & scenario testing.** Rule changes run against synthetic datasets pre-deployment, to catch side effects a human reviewer would miss.
- **Semantic versioning.** Rule sets and schemas are versioned explicitly. Consuming microservices declare which version of a policy or schema they depend on, so a rule change cannot silently break a downstream consumer.

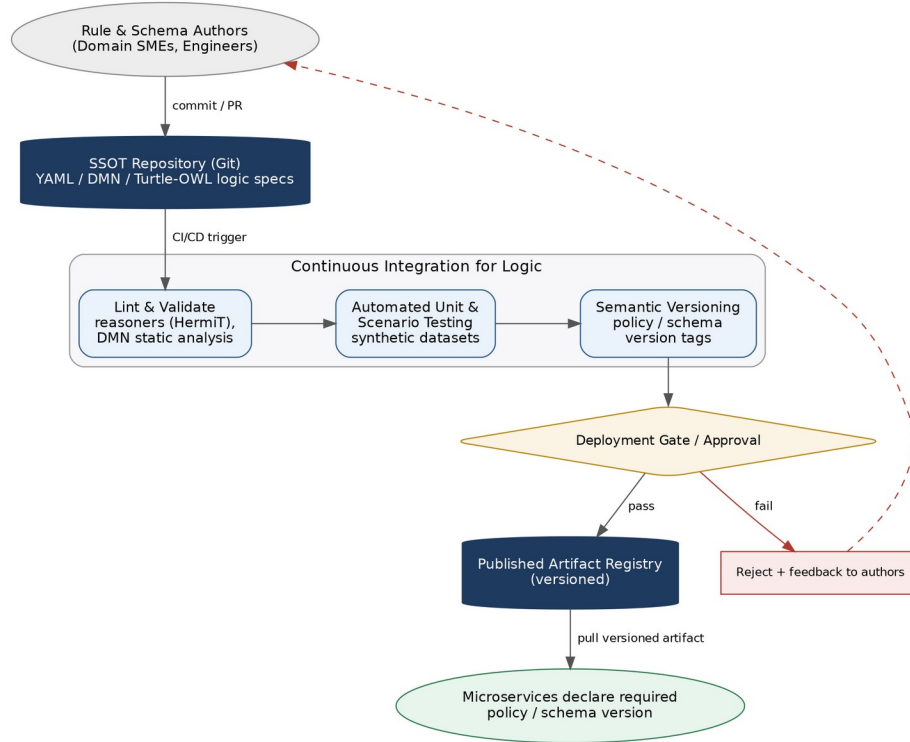


Figure 2. The GitOps loop for logic: commit → CI validation → gated release → versioned consumption, with rejected changes routed back to the author.

3. The Modern Pattern: Declarative Spec + Generated Executables

Rather than rewriting legacy applications to “speak the same language,” the architecture is **declarative-first**: logic is written once, and a build pipeline compiles it down to whatever each target runtime needs.

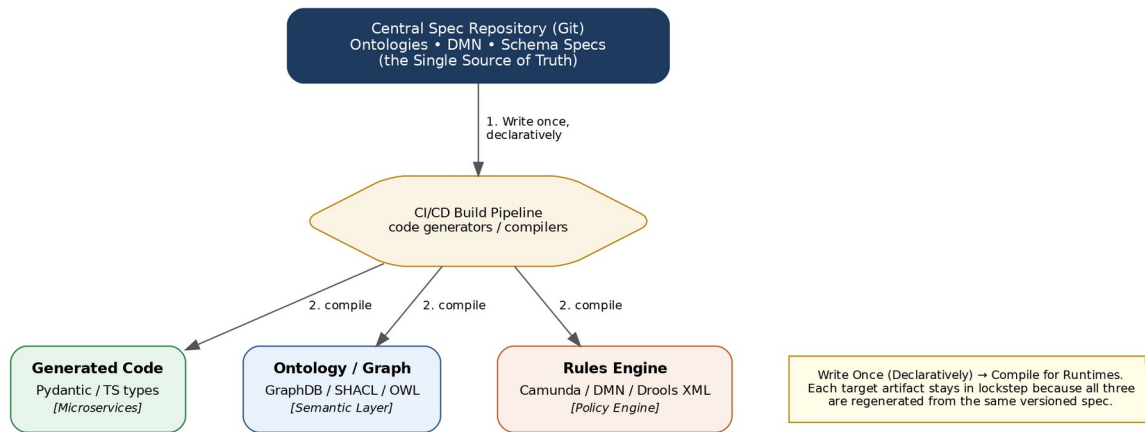


Figure 3. One declarative spec, three generated runtime artifacts, one CI/CD compiler in between.

How it works

- **Write once, declaratively.** Schema constraints, taxonomy, and core decisions are defined in standardized formats — OpenAPI/JSON Schema for constraints, DMN for rules.
- **Compile for runtimes.** Code generators in CI/CD produce the target artifacts on every merge:
 - TypeScript/Python interfaces for microservices
 - SHACL/OWL files for the semantic graph store
 - DMN/Drools XML for the rules-engine runtime

Because all three artifacts regenerate from the same versioned spec on every build, they cannot drift out of sync with each other — drift becomes a build failure, not a production incident.

Runtime View: The Decision API

At request time, callers don't talk to the rules engine, the ontology store, or a legacy stored procedure directly. They talk to a **standardized Decision API** that resolves the caller's declared policy version, validates the request against the current schema, and routes to whichever backend currently owns that decision — including a wrapped legacy engine that hasn't been migrated yet.

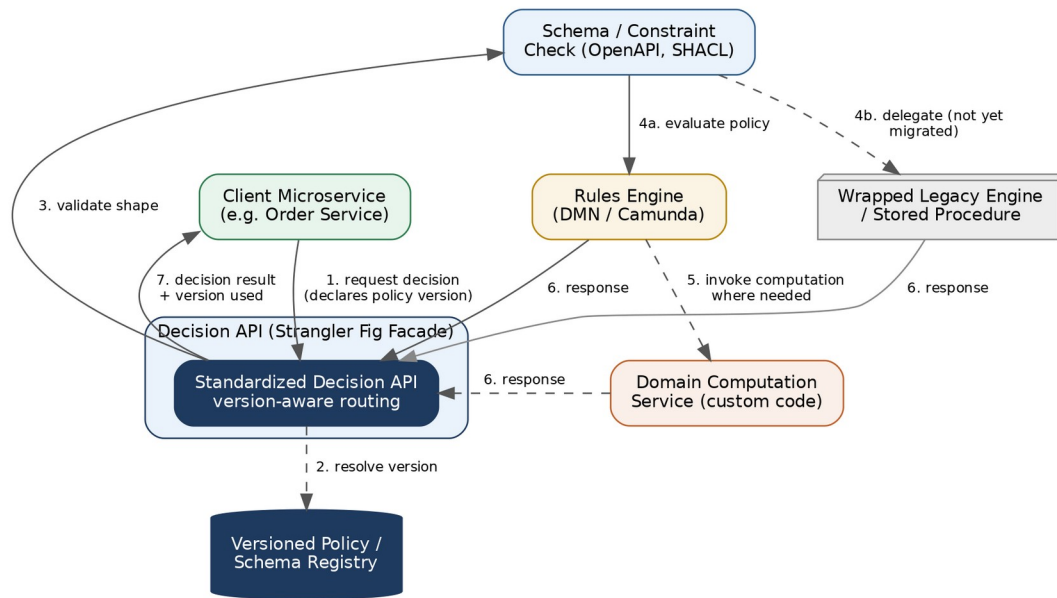


Figure 4. A client microservice never calls the rules engine, ontology store, or legacy procedure directly — it calls the Decision API, which is free to route to legacy or modern backends without the client knowing which.

This indirection is what makes the migration in Section 4 possible: the facade lets legacy and modernized decision logic sit behind one interface, so callers never need to change when the backend behind that interface changes.

4. Practical Migration Blueprint for Legacy Systems

For logic trapped in stored procedures, hardcoded Java/COBOL, and legacy rules engines, migrate in three phases rather than attempting a rewrite:

Phase 1 — Logic Discovery & Extraction

- **Inventory & audit.** Map existing logic with tracing tools and static analysis. Log every point where a business decision is made.
- **Classify.** Sort each finding into Constraint, Rule, or Computation using the framework in Section 1.

Phase 2 — Decouple via API Boundaries (Strangler Fig)

- **Wrap, don't rewrite.** Put legacy rules engines and stored procedures behind the standardized Decision API from Section 3 before touching their internals.
- **Extract incrementally.** Pull hardcoded logic out of application code into externalized configuration or DMN files piece by piece, re-pointing the facade as each piece lands.

Phase 3 — Centralize Schemas First

- **Standardize data shapes.** Unify data definitions across teams via a common ontology or schema registry before consolidating rules.
- **Consolidate rules second.** Once the underlying data definitions (constraints) are unified, reconciling the business rules that operate on that data becomes dramatically simpler — most apparent rule conflicts turn out to be data-shape conflicts in disguise.

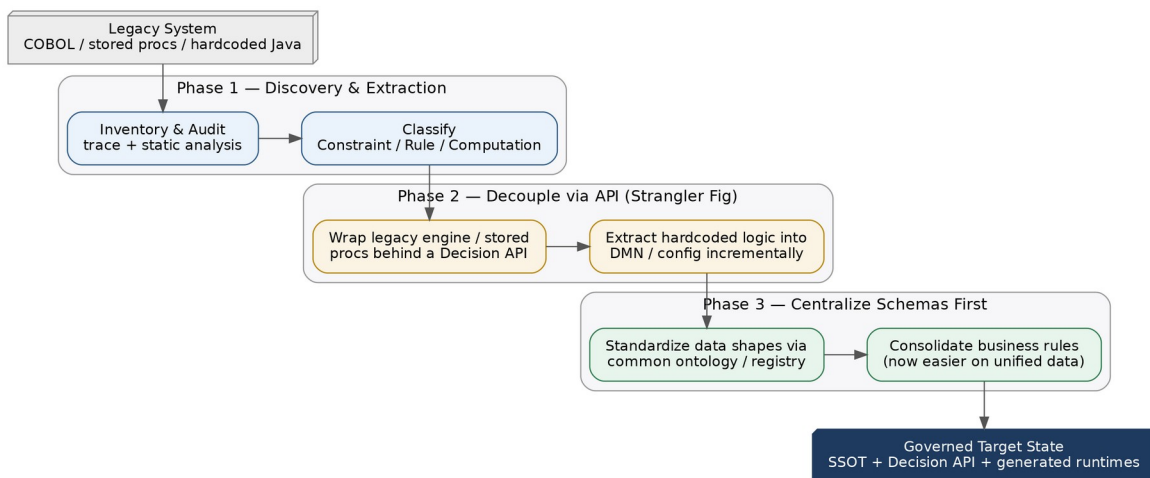


Figure 5. Three-phase strangler-fig migration: discover and classify, decouple behind a Decision API, then centralize schemas before consolidating rules.

Key Pitfalls to Avoid

Recommended Next Steps

- **Stand up the SSOT repository and CI pipeline first** — before extracting a single rule. Governance has to exist before there's anything worth governing.
- **Pilot on one bounded decision** (e.g., discount eligibility) to prove the classify → spec → compile → deploy loop end-to-end, including the Decision API facade.
- **Inventory before you migrate.** Run Phase 1 across the legacy estate broadly before committing engineering time to Phase 2 extraction — classification changes the shape of the migration plan.
- **Make version declaration mandatory** for any service consuming a policy or schema from day one, so the versioning discipline never has retrofitting debt.